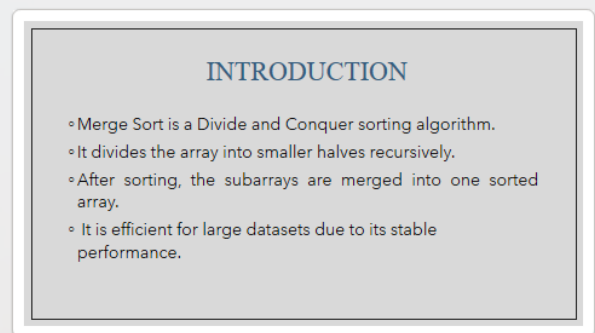
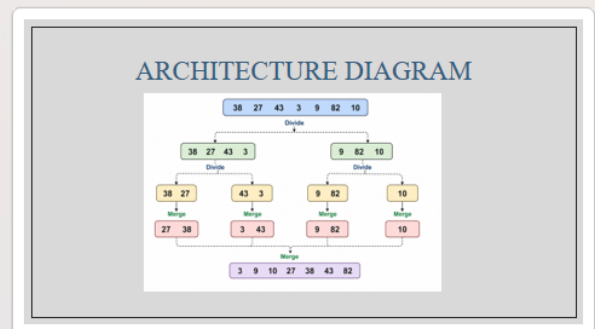
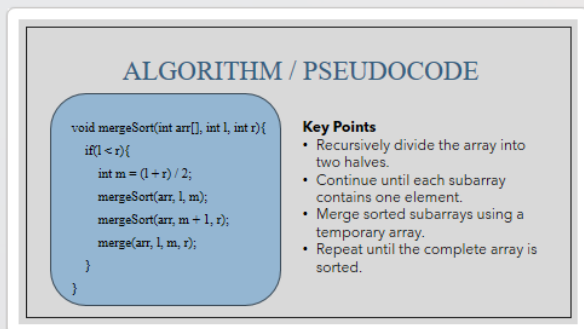




1



2



PROBLEM STATEMENT

- Analyze the time complexity of Merge Sort.
- Evaluate the auxiliary space required during execution.
- Compare Merge Sort with the in-place Quick Sort algorithm.
- Interpret the trade-off between execution time and memory usage.

COMPLEXITY ANALYSIS

Algorithm	Best	Average	Worst	Reason
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Always divides and merges the same way.
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	Performance depends on pivot selection.
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Faster if the array is already sorted.

- Merge Sort requires $O(n)$ auxiliary memory.
- Time complexity remains $O(n \log n)$ in all cases.
- Quick Sort uses less memory but has $O(n^2)$ worst-case time.
- Merge Sort offers stable performance with higher space usage.

OUTPUT

Input

◦ 38 27 43 3 9 82 10

Output

◦ 3 9 10 27 38 43 82

Points

- Input array is successfully sorted.
- Relative order of equal elements is preserved.
- Merge operation combines sorted halves correctly.
- Output verifies the correctness of Merge Sort.

CONCLUSION

- Merge Sort provides consistent $O(n \log n)$ performance.
- Auxiliary space $O(n)$ is needed for merging.
- Quick Sort is more memory efficient but less predictable.
- Choice depends on the trade-off between speed and memory.